Exercise 10

I2C / TWI communication basics:

Master transmitter – Slave receiver modes (Microchip Studio)

In this task, you will build a communication between Arduino MEGA and Arduino UNO using I2C/TWI (2 wire Serial Interface). Arduino MEGA will act as master, so it initiates and terminates a data transmission, and also generates the SCL clock. Arduino UNO will act as slave, which is addressed by Master.

Build the circuit using these components:

- Arduino MEGA
- Arduino UNO
- Wires

Connect Arduino MEGA to Arduino UNO via I2C pins, SDA for data and SCL for clock. Find the corresponding pins in Arduino by checking the Pinout diagram.
Connect the ground of Arduino MEGA to the ground of Arduino UNO.

Code for Arduino MEGA (master):

1. Build communication between your computer and Arduino MEGA using USART. The step is similar to the Exercise 8.

2. Include the 7-bits of slave address, for example:
```
#define SLAVE_ADDRESS 0b1010111 // 87 as decimal. You can try another address but ensure that
master and slave have the same address.
```

3. In the main (void):
   - After you initialize the UART and redirect the `stdout` and `stdin`, initialize the TWI.
     The SCL period is controlled by settings in the TWI Bit Rate Register (TWBR) and the Prescaler bits in the TWI Status Register (TWSR). Set the SCL frequency to 400 kHz and set the TWI status register prescaler (TWPS) to 1. Therefore, you can calculate TWBR using the equation in the datasheet p. 242.
```
TWBR = 0x03; // TWI bit rate register.
TWSR = 0x00; // TWI status register prescaler value set to 1
```

   - Set TWEN to enable TWI. See datasheet p. 248.
```
TWCR |= (1 << TWEN); // The TWEN must be set to enable the TWI, datasheet p. 248
```

   - Create variable of data array that will be transferred to the slave, and received from the slave:
```
unsigned char twi_send_data[20] = "master to slave\n"; //Instead of SPI, now is TWI
char test_char_array[16]; // 16-bit array, assumes that the int given is 16-bits
uint8_t twi_status = 0;
```

4. In the void loop:
   Follow the steps in the **datasheet p. 246**:

   Step 1: Send START condition.
```
TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
```

Step 2: Wait for TWINT Flag set. This indicates that the START condition has been transmitted.

```
while (!(TWCR & (1 << TWINT)))
{;}
```

Step 3a: Read status from TWI status register. Mask the prescaler bits using 0xF8. See datasheet p. 262. You can check the status bits by printing to the UART for monitoring purposes. Use *itoa* to convert integer to string.

```
twi_status = (TWSR & 0xF8);

itoa(twi_status, test_char_array, 16);
printf(test_char_array);
printf(" ");
```

Step 3b_1: Load SLA_W into TWDR register.

```
TWDR = 0b10101110; //SLA_W --> 7-bit slave address and 1 write bit//p. 247, p. 263
// Slave address + write bit '0' as an LSB
```

Step 3b_2: Clear TWINT bit in TWCR to start transmission of address.

```
TWCR = (1 << TWINT) | (1 << TWEN);
```

Step 4: Wait for TWINT Flag set. This indicates that the SLA+W has been transmitted, and ACK/NACK has been received.

```
while (!(TWCR & ( 1<< TWINT)))
{;}
```

Step 5a: Check value of TWI Status Register. Mask prescaler bits using 0xF8. You can check the status bits by printing to the UART for monitoring purposes. Use *itoa* to convert integer to string.

```
twi_status = (TWSR & 0xF8);

itoa(twi_status, test_char_array, 16);
printf(test_char_array);
printf(" ");
```

Step 5b, Task 1: Load DATA into TWDR register.

```
for(int8_t twi_data_index = 0; twi_data_index < sizeof(twi_send_data); twi_data_index++)
{
// Load data into TWDR
TWDR = twi_send_data[twi_data_index];
```

Step 5b, Task 2: Clear TWINT bit in TWCR to start transmission of data.

```
// Clear TWINT bit in the TWCR to start transmission of data
TWCR = (1 << TWINT) | (1 << TWEN);
```

Step 6: Wait for TWINT Flag set. This indicates that the DATA has been transmitted, and ACK/NACK has been received.

```
while (!(TWCR & ( 1<< TWINT)))
{;}
```

Step 7a: Check value of TWI Status Register. Mask prescaler bits using 0xF8. You can check the status bits by printing to the UART for monitoring. Use *itoa* to convert integer to string.

```
twi_status = (TWSR & 0xF8);

itoa(twi_status, test_char_array, 16);
printf(test_char_array);
printf(" ");

}  //this is end of for command in the step 5b.
```

Step 7b: Transmit STOP condition.

```
TWCR = (1 << TWINT) | (1 << TWEN) |(1 << TWSTO);

printf("\n");
```

You can put delay here, for example 1 second.

Code for Arduino UNO (slave):

1. Build communication between your computer and Arduino MEGA using USART. The step is like the Exercise 8.
2. Include the 7-bits of slave address, for example:
   ```
   #define SLAVE_ADDRESS 0b1010111 // 87 as decimal. This address must be the same as master address.
   ```
3. In the main (void):
   - After you initialize the UART and redirect the `stdout` and `stdin`, initialize the TWI Slave. See datasheet p.230.
     Step 1:
     ```
     TWCR |= (1 << TWEA) | (1 << TWEN);
     TWCR &= ~(1 << TWSTA) & ~(1 << TWSTO);
     ```
   - Set the upper 7 bits of slave address that will respond when called by a Master to the register TWAR. If the LSB is set, the TWI will respond to the general call address (0x00), otherwise it will ignore the general call address. See datasheet p. 223.
     ```
     TWAR =  0b10101110; // 7-bit slave address and 1 write bit (LSB)
     ```
   - Create variable of data array that will be transferred to the master, and received from the master:
     ```
     char twi_receive_data[20]; // Use TWI instead of SPI
     char test_char_array[16]; // 16-bit array, assumes that the int given is 16-bits
     uint8_t twi_index = 0;
     uint8_t twi_status = 0;
     ```
4. In the while loop:
   Follow steps in **datasheet p. 216** starting from step 2 to step 4.

   Step 2: Wait for TWINT Flag set. This indicates that the START condition has been transmitted.
   ```
   while(!(TWCR & (1 << TWINT)))
   {;}
   ```

   Step 3a: Check value of TWI status register. Mask prescaler bits.
   ```
   twi_status = (TWSR & 0xF8);
   ```

Step 3b: Clear TWINT bit in TWCR to start transmission of address.

```
TWCR |=  (1 << TWINT) | (1 << TWEA) | (1 << TWEN);
```

Step 4: Wait for the TWINT Flag set.

```
while(!(TWCR & (1 << TWINT)))
{;}
```

Step 5: Check value of TWI status register by reading the status bits from the status register TWSR. Mask prescaler bits using 0xF8.

```
twi_status = (TWSR & 0xF8);
```

Step 6: Follow table 22-4 in the datasheet p. 224 for status code for slave receiver mode.

- If the status codes are "0x80" or "0x90", it means that data has been received and acknowledgement has been transmitted to master. Then read data register TWDR to your char list.

```
if((twi_status == 0x80) || (twi_status == 0x90))
{
   twi_receive_data[twi_index] = TWDR;
   twi_index++;
}
```

- If the status code is "0x88" or "0x98", it means that data has been received and there is no acknowledgement transmitted to the master. Then, read data register TWDR to your char list.

```
else if((twi_status == 0x88) || (twi_status == 0x98))
{
   twi_receive_data[twi_index] = TWDR;
   twi_index++;
}
```

- If status code is "0xA0", it means that either STOP conditions or repeated START has been transmitted to the slave. When this occurs, clear the interrupt flag TWINT.

```
else if(twi_status == 0xA0)
{
   TWCR |= (1 << TWINT);
}
```

- Print the message using USART in **PuTTY**, when message has been received.

```
if (20 <= twi_index)
{
   printf(twi_receive_data);
   twi_index = 0;
}
```

NOTE: After you build the code, upload and print out the data using PuTTY. You need to open two PuTTY windows, one for Arduino MEGA and one for Arduino UNO.

When you open PuTTY for Arduino UNO, you will see the message from Master (Arduino MEGA), for example "master to slave"

```
COM5 - PuTTY
master to
lave
master to slave
master to slave
master to slave
master to slave
master to slave
aster to slave
master to slave
master to slave
master to slave
master to slave
maste
to slave
master to slave
master to slave
master to slave
master to slave
master to
lave
master to slave
master to slave
```

When you open PuTTY for Arduino MEGA, you will see message from slave (Arduino UNO):

Read Table 24-2 in ATmega2560 datasheet p. 249 to understand these messages.

```
COM4 - PuTTY
8 18 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28
8 18 28 28
28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28
8 18 28 28 28 28 28 28 28
28 28 28 28 28 28 28 28 28 28 28 28 28
8 18 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28
28 28 28 28 28 28 28 28
8 18 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28
28 28 28
8 18 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28
8 18 2
8 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28
8 18 28 28 28 28 28 2
8 28 28 28 28 28 28 28 28 28 28 28 28 28 28
8 18 28 28 28 28 28 28 28 28 28 28 2
8 28 28 28 28 28 28 28 28 28
8 18 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 2
8 28 28 28 28
8 18 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28
8
18 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28
8 18 28 28 28 28
28 28 28 28 28 28 28 28 28 28 28 28 28 28 28 28
```

To understand the value printed in the window, check ATmega 2560 datasheet p. 249 in Table 24-2. Status codes for Master Transmitter Mode.